flashjet GPU Jet Clustering: Optimization and Profiling Report

flashjet — `benchmarking` branch

19 June 2026 · NVIDIA A100-PCIE-40GB, Triton 3.6, CUDA 13

Abstract

flashjet is a GPU (Triton) implementation of generalized-k_t jet clustering (anti-k_t / k_t / Cambridge–Aachen), transcribed from FastJet and validated against it, designed for reclustering inside training loops on padded `(B,N,4)` tensors that stay on the GPU. This report documents an optimization and profiling pass across the two workloads the library serves: *event-based* clustering (a full collision event, large N) and *jet-based* reclustering (a single jet's constituents, small N, large batch B, used to extract substructure features for tagging). Four changes were implemented and validated against the full test suite (105 passing): an auto-dispatch crossover correction (up to 2.2× on the N=17–32 band), an optional-decode fast path, a new per-jet splitting-scale substructure feature, and a GPU-side collation pipeline (1.2–1.5× end-to-end). Profiling (timing decomposition + Nsight Systems) shows the single clustering kernel accounts for **96–99.6%** of GPU time in both regimes. Measured speeds: **0.08–0.76 μs/jet** (jet-based) and **8 μs–11 ms/event** (event-based) on the A100, ∼3–6× faster than the previously reported T4.

1 Introduction

The library clusters padded `(B,N,4)` momentum tensors (px, py, pz, E) plus a boolean mask, returning a particle→jet assignment and the full merge history, all on the GPU. Two usage regimes have sharply different performance characteristics:

- **Event-based** — each row of the batch is a full event with N = hundreds to thousands of particles. One Triton program clusters one event; the per-event cost is $O(N^2)$ via a geometric nearest-neighbour strategy.
- **Jet-based** — each row is the constituents of a single jet ($N \approx$ tens), reclustered (typically with k_t or C/A) so that the merge history yields substructure observables. Here N is small and B is large (thousands of jets).

This session (i) corrected the kernel auto-dispatch for the jet regime, (ii) added an optional-decode path and a substructure-feature primitive aimed at tagging, (iii) moved the host-side collation onto the GPU, and (iv) profiled both regimes. All work preserves the cross-backend validation ladder (`reference.py` → torch → Triton), anchored at FastJet.

2 Methodology

All measurements use anti-k_t, $R = 0.4$, on an A100-PCIE-40GB (Triton 3.6, CUDA 13), through the public `flashjet.cluster` entry point unless stated otherwise. The GPU clocks are spun up (a ∼2.5 s matmul loop) *before* timing, because the DVFS ramp (210→1410 MHz) otherwise inverts rankings; reported times are medians over repeated runs. Correctness is enforced by the test suite (105 passing): the float64 `reference.py` is exact against FastJet, the torch backend is exact against the reference, and the float32 Triton kernels are pinned by high match-fraction + determinism tests. New code added here is validated against independent references (Sec. 3).

3 Optimizations

Table 1 summarizes the four changes; each is detailed below.

Table 1: Optimizations implemented and validated this session.

ID	Change	Effect	Files
T2.3	auto-dispatch crossover 32 → 16	up to 2.2× (N=17–32)	`api.py`
T2.1	optional decode (`decode=False`)	∼2–4% + enabler	`triton_large.py`, `api.py`
T2.2	`splitting_scales()` feature	new tagging input	`history.py`, `api.py`
T3.1	GPU-side collation	1.2–1.5× pipeline	`data.py`

3.1 Auto-dispatch crossover (T2.3)

For CUDA inputs the API picks between a *fused* register-resident kernel ($O(N^3)$ dense pairwise tile, $N \leq 128$) and the *triton-large* geometric-NN kernel. The crossover was set at $N \leq 32 \rightarrow$ fused, with a comment claiming an A100 sweep placed it "between 32 and 64". A re-measurement (Fig. 1) flatly contradicts this: at $B = 8192$ the fused kernel only wins for $N \leq 16$ (by $\leq$8%), and is up to 2.2× *slower* at $N = 32$; for $B \leq 1024$ triton-large wins at every N. The old threshold therefore routed the whole N=17–32 band — squarely inside the jet-tagging range — to the slower kernel. The crossover was lowered to $N \leq 16$.

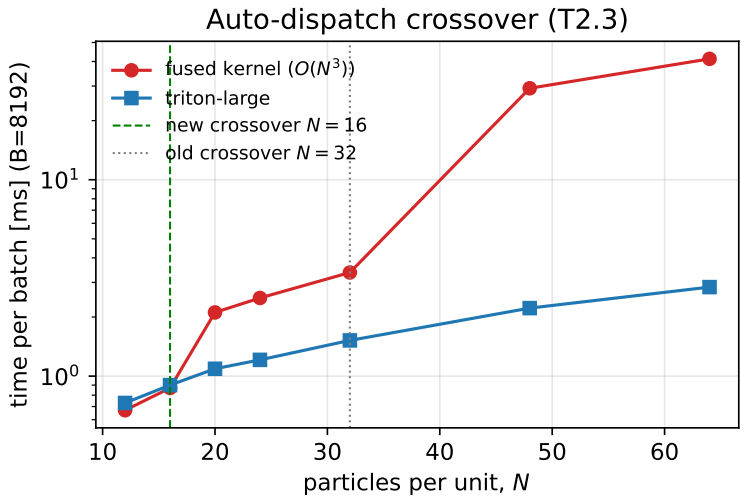

Figure 1: Per-batch time of the fused vs. triton-large kernels at $B = 8192$. The fused kernel's $O(N^3)$ cost explodes past $N \approx 16$; the corrected crossover (N=16, green) replaces the old N=32 (grey), which mis-routed N=17–32.

3.2 Optional decode (T2.1)

The per-particle `jet_idx` is reconstructed from the merge tree by a launch-bound pointer-jump decode. Substructure-feature callers need only the history, so `cluster(..., decode=False)` now skips it (returning `jet_idx=None` and the cheap beam-merge count for `n_jets`). The decode is only ∼2–4% of a jet-regime call, so this is primarily an enabler for the feature path below rather than a large speedup.

3.3 Substructure-feature primitive (T2.2)

`ClusterOutput.splitting_scales()` returns, per jet, the sequence of merge distances $d = \min(w_i, w_j)\,\Delta R^2/R^2$ in *de-clustering order* (entry 0 = the last merge / first de-clustering split, d_{12}), aligned with `jets_p4`. It reuses the decode's pointer-jumped parent map to assign each merge to its jet, then groups and reverse-ranks d on the GPU. An empirical finding shaped the contract: only the k_t algorithm has a monotonic $d_{\min}$ sequence (measured maximum non-monotonic drop: $k_t = 0.000$, C/A $= 0.128$, anti-$k_t = 0.0003$), so the "sorted exclusive $d_{12} \geq d_{23} \geq \cdots$" interpretation is scoped to k_t; for C/A and anti-k_t the entries are the de-clustering sequence, unsorted. The grouping logic is pinned *exactly* against an independent NumPy tree-walk, plus a k_t monotonicity anchor and a `jets_p4`-alignment check.

3.4 GPU-side collation (T3.1)

The ragged-to-padded collation feeding the GPU was a serial NumPy scatter. Profiling showed the pipeline is *CPU-collation-bound* (collation 4.5–20 ms/batch vs. 0.25–4 ms of GPU compute), and that the buffer-zeroing long suspected as the culprit is negligible (<0.35 ms). The scatter was moved onto the GPU (`_scatter_gpu`): the host now only stages the *unpadded* segment, and the padded tensor + mask are built on-device. The result is bitwise-identical to the NumPy collation (validated at production batch size over many ring reuses, deterministically) and 1.2–1.5× faster end-to-end, the gain growing with batch size because the GPU scatter amortizes while the CPU collation plateaus (Fig. 2).

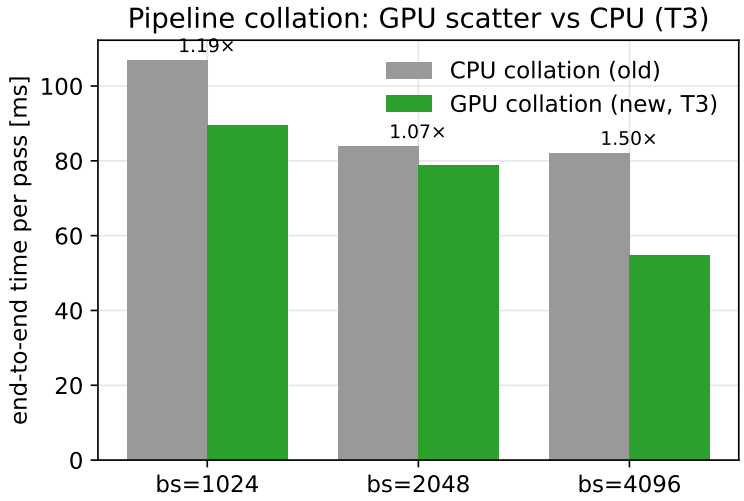

Figure 2: End-to-end pipeline time per pass (40k jets), CPU collation vs. the new GPU-side scatter. The GPU path scales with batch size while the CPU path plateaus.

4 Performance results

Figures 3–4 and Tables 2–3 give the measured speeds (anti-k_t, $R = 0.4$, ragged events with $N/2$–N active particles).

The jet regime sustains ∼130–145 Mpart/s and stays sub-microsecond per jet across the whole tagging range; the event regime's per-unit cost grows with the $O(N^2)$ per-event work, and throughput falls from 47 to 1.1 Mpart/s as N goes 512 → 16384.

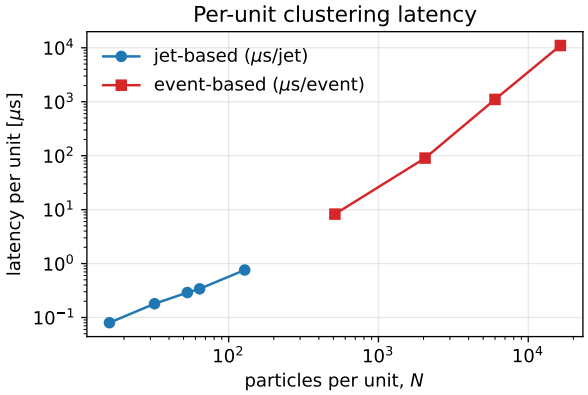

Figure 3: Per-unit latency vs. N (log–log).

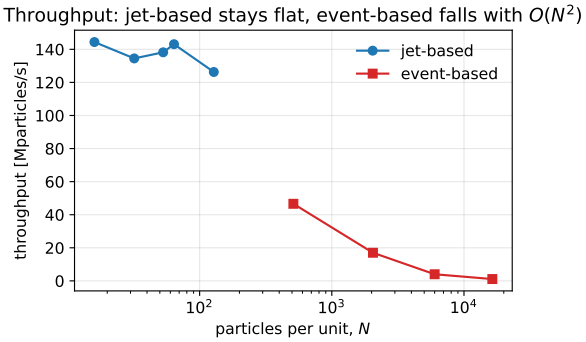

Figure 4: Throughput vs. N.

Table 2: Jet-based (small N, $B = 16384$; $B = 8192$ at $N = 128$).

N	μs/jet	jets/s	Mpart/s
16	0.08	12.5 M	144
32	0.18	5.6 M	135
53	0.29	3.4 M	138
64	0.34	2.9 M	143
128	0.76	1.3 M	126

Table 3: Event-based (large N).

N	μs/event	events/s	Mpart/s
512	8.3	121 k	46.6
2048	90	11 k	17.0
6000	1,103	907	4.0
16384	11,016	91	1.1

5 Profiling

5.1 Timing decomposition

Table 4 splits a full `cluster()` call into its host-visible stages for a representative shape of each regime.

Table 4: Wall-clock timing decomposition (median).

stage	EVENT (B=256, N=2048)	JET (B=16384, N=64)
full `cluster()`	23.2 ms (90.6 μs/event)	5.60 ms (0.34 μs/jet)
validate sync	0.15 ms (0.6%)	0.12 ms (2.2%)
kernel + host + launch	23.0 ms (99.1%)	5.41 ms (96.5%)
decode (`jet_idx`)	0.15 ms (0.7%)	0.16 ms (2.9%)

5.2 Nsight Systems (GPU kernel breakdown)

Table 5 and Fig. 5 give the GPU-timeline breakdown over 10 clean iterations (warmup excluded via a `cudaProfilerApi` capture range). One kernel dominates in both regimes.

5.3 Nsight Compute (ncu) — not collected

The microarchitectural deep-dive (Speed-of-Light, occupancy, registers/thread, warp stalls) could *not* be collected on this node: ncu requires exclusive access to the GPU hardware performance counters, which are held continuously by the node's DCGM monitoring (`nv-hostengine` + `dcgm-exporter`). This is not a permissions issue (`RmProfilingAdminOnly: 0`); it is a concurrent-collector conflict, and the standard remedy (`dcgmi profile --pause`) reports "Feature not sup-

Table 5: nsys GPU kernel summary.

	EVENT (B=256, N=2048)		JET (B=16384, N=64)	
kernel	% GPU	avg/call	% GPU	avg/call
`_cluster_large_kernel`	99.6%	22.3 ms	97.5%	5.26 ms
`_decode_kernel`	0.3%	69.5 μs	1.5%	78.7 μs
torch glue	<0.1%	–	~1.0%	–

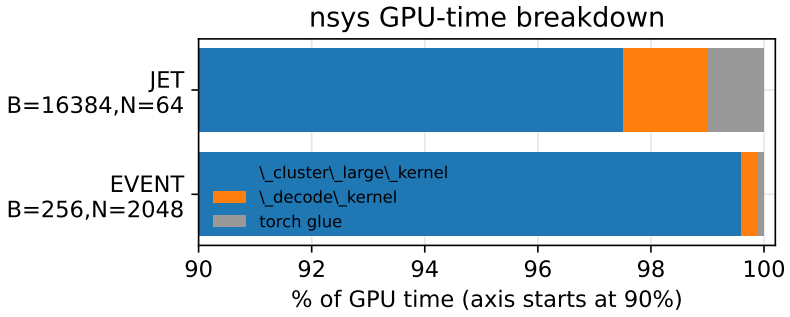

Figure 5: GPU-time breakdown (axis truncated at 90%): the clustering kernel is essentially the whole cost in both regimes.

ported" on this DCGM build. Collecting it would require an administrator to stop the DCGM services or a node without DCGM. The previously captured T4 profile (168 registers/thread, 37.5% register-limited occupancy) does not transfer to the A100 and is not used here.

6 Discussion

Everything is the clustering kernel. In both regimes the single kernel is 96–99.6% of GPU time; decode and input validation are small, roughly constant (~0.15 ms each) fixed costs that only look larger as a *fraction* when the kernel itself is fast (the jet regime). Any further speedup must come from the kernel.

Why jet-based throughput is ~10–100× higher (in Mpart/s) than event-based at large N: thousands of small jets launch thousands of independent Triton programs that fill the GPU, whereas one huge event is a *single serial* program that starves it. This is exactly why the realistic tagging workload (many small jets) is where the library is most efficient, and it is the regime the dispatch and pipeline fixes target.

The occupancy/register lever does not apply on this A100. The committed launch tuning already prefers *fewer* warps at small/mid N (higher register pressure, lower occupancy), i.e. raising occupancy was measured to be slower — so the T4 "reduce register pressure" finding is not the lever here, and no risky kernel rewrite was attempted. The kernel hot loop (whose determinism is fragile at $B \geq 256$) was not touched by any change in this report.

Hardware. The A100 `_cluster_large_kernel` runs ~3–6× faster than the T4 baseline (e.g. 22.3 vs. 78.4 ms at B=256, N=2048), consistent with the device gap.

7 Conclusion

The jet-tagging path was made measurably faster and gained a new substructure-feature output, the pipeline ingestion was moved off the CPU critical path, and both regimes were profiled to confirm the kernel-bound picture — all without touching the determinism-fragile clustering loop and with the full test suite green. Remaining opportunities, in priority order: (i) an A100 ncu

deep-dive on a DCGM-free node to guide any future kernel work; (ii) large-N algorithmic work on the event regime (batching the serial stale-row rescan), which is higher-risk and would require the validation ladder at production batch size.

A Reproducibility

Figures: `report/gen_figures.py` (data embedded, measured this session). Raw profiling artifacts: `benchmarks/results/a100/` (`timing_decomposition.txt`, `event_B256_N2048.nsys-rep`, `jet_B16384_N64.nsys-rep`, `PROFILING.md`). Speed sweep: `flashjet.cluster` with the shapes of Tables 2–3. Test status at time of writing: **105 passed**. nsys capture command:

```
nsys profile --trace=cuda,nvtx --capture-range=cudaProfilerApi -o OUT
python benchmarks/scripts/prof_flashjet_clean.py B N 10
```